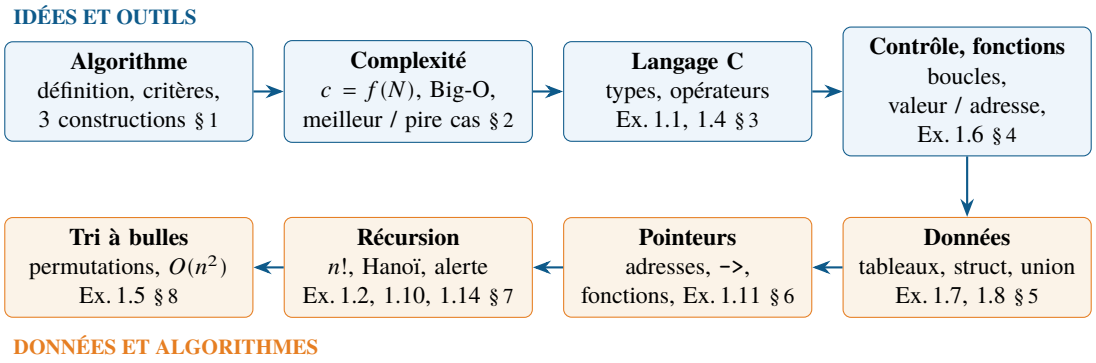
Algorithmes — Fiche de révision du chapitre 1

algorithme · complexité · langage C · pointeurs · récursion · tri à bulles

Date du TE : pas encore annoncée. Chaque TE compte pour 2/5 de la note (Note(CC) = (2 (TE1 + TE2) + Ra)/5).
Sources : diapos Ch1 de R. Scherwey et D. Bullot (Introduction, Complexité, Langage C basics, supplément fonctions, Pointeurs, Récursion), énoncés 1.1 à 1.14, notes 1.2 et 1.7 et code de Philippe.
Code C compilé (gcc 13, `-std=c11 -Wall -Wextra -Werror -pedantic`) et testé : `scripts/fiche_chapitre1_calculs.py` (86 contrôles C + 237 contrôles Python).

Carte du chapitre : ce qui s'enchaîne avec quoi



Comment utiliser cette fiche

Chaque section suit le même ordre : **la notion** (diapo), **le code** (extrait compilé et testé, numéro d'exercice entre parenthèses) et **le piège**. Les valeurs chiffrées sortent du script de contrôle. À relire la veille du TE : la **méthode** (§ 9), le **mémo des pièges** (§ 10) et l'**auto-test** (§ 11).

L'idée directrice en une phrase

Un algorithme transforme une **entrée** en **sortie** par une suite *finie* et *non ambiguë* d'opérations (séquence, décision, répétition) ; sa **complexité** dit comment son coût grandit avec la taille N des données ; le C fournit les briques (types, structures, **pointeurs** = adresses) et la **récursion** ramène un problème à des cas plus petits, jusqu'au cas trivial.

Ce qui manque dans l'arbre

Les énoncés **1.9, 1.12 et 1.13** n'ont pas été déposés. Dans `Code/`, les fichiers 1.1, 1.3 et 1.4 sont vides ; le 1.5 est rendu dans `TP-Rapports/Ex1-5/`. Le brouillon 1.14 n'appelle que 7 personnes sur 14 (§ 7 donne une version testée). Aucun ancien TE ni corrigé officiel : la **convention de coût de l'exemple 2** (§ 2) est *proposée*, à confirmer en classe.

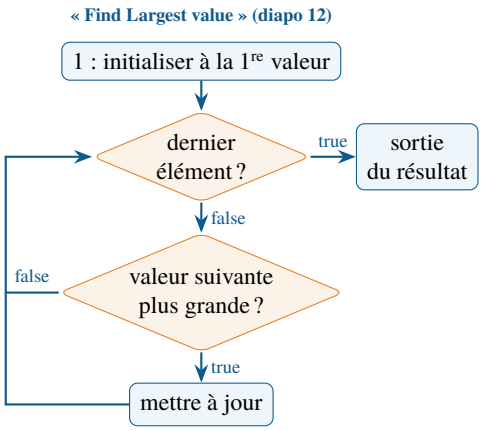
1. Algorithme : définition, critères et démarche

Définition : une suite **finie** et **non ambiguë** d'opérations qui résout un problème ou donne un résultat ; en temps fini, d'une *précondition* (situation de départ) à une *postcondition* (résultat attendu). Le mot vient du mathématicien persan al-Khwarizmi (vers 780). Un algorithme est **indépendant du langage** : l'énoncé fixe la relation entrée → sortie (« Input → Data Manipulation → Output » ; ex. trier 7, 5, 9, 6, 2, 1 en 1, 2, 5, 6, 7, 9).

Critères indispensables : des valeurs d'**entrée** et de **sortie** ; il **se termine** ; il est **déterministe** (fiable). **Souhaités** : efficace (durée déterminable), simple et compréhensible, maintenable, modulaire, flexible, convivial, robuste.

Démarche : analyser la fonctionnalité → la décomposer « sur papier » (organigramme) → planifier l'implémentation.

Trois constructions suffisent : **séquence** (instructions à la suite), **décision** (`if`, `if-else`, `switch`, `?:`), **répétition** (`do-while` : test *après*, au moins un passage ; `while` / `for` : test *avant*, zéro passage possible).

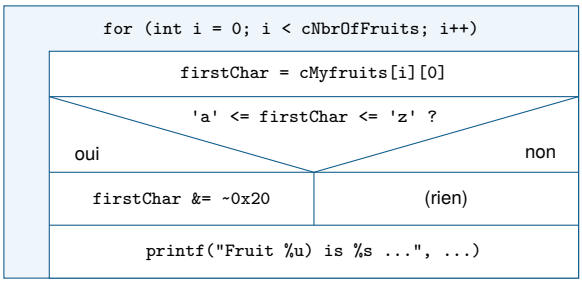


```c
#include <stddef.h>
#include <stdint.h>

/* « Find Largest value » (diapos 9-12), n >= 1 */
int32_t findLargest(int32_t const table[], size_t n)
{
    int32_t largest = table[0];      /* 1 : 1re valeur */
    for (size_t i = 1u; i < n; i++) {   /* répétition */
        if (table[i] > largest) {       /* décision */
            largest = table[i];         /* mise à jour */
        }
    }
    return largest;                     /* sortie */
}
```

Déroulé sur 7, 5, 9, 6, 1 : 7, 7, 9, 9, 9 ⇒ 9. **Initialiser à la 1re valeur**, pas à 0 : sinon $\{-4, -2, -7\}$ rendrait 0 au lieu de -2 (testé). « Potentiel d'optimisation ? » (diapo 12) : chaque valeur doit être lue au moins une fois, on reste en $O(n)$; on évite seulement de relire la 1re (boucle dès `i = 1`).

Ex. 1.3 — Nassi-Shneiderman de la boucle des fruits (`firstChar &= ~0x20` met en majuscule, § 3) : la boucle est un bloc qui *englobe* son corps, la décision un triangle oui / non.



Le **diagramme de flux** de la même boucle a la forme ci-dessus (test de boucle, puis décision).

2. Complexité : coût en temps et en mémoire

Deux questions : *calculabilité* (existe-t-il un algorithme ?) et *complexité* (quel est le meilleur, à quel coût ?). Thèse de Church–Turing : une fonction est calculable par algorithme si et seulement si une **machine de Turing** la calcule.

Machine de Turing : bande infinie de cases (symboles d'un alphabet), tête de lecture / écriture qui se déplace d'une case, registre d'état, table de transitions (selon état et symbole lu : écrire ou non, bouger à gauche / à droite / rester, changer d'état).

Coûts : le **temps** (nombre de transitions) et la **mémoire** (nombre de cases utilisées). Ils dépendent de la taille N des données : trier 10 nombres ≠ trier 1 000 000.

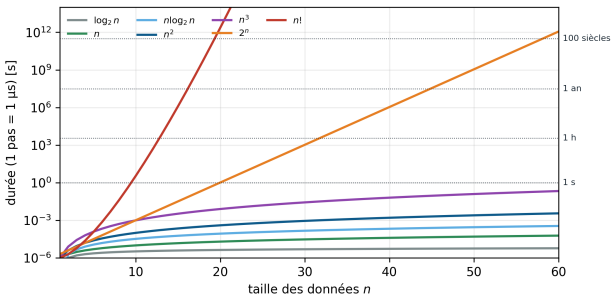
$$c = f(N) \qquad \text{(coût en temps } c_t \text{ ou en mémoire } c_s\text{)}$$

Exemple 1 (somme d'un tableau), toutes les instructions coûtent t : `s := 0` une fois, puis N tours de 4 instructions (incrément du compteur, accès au tableau, addition, affectation).

$$c_t(N) = t + 4N\,t = (1 + 4N)\,t \qquad c_t(10) = 41\,t$$

Indépendant de l'ordre des nombres. Mémoire (proposé) : $c_s(N) = N + 2$ cases (tableau, `s`, `i`).

Coût	Big-O	Exemple (diapo 8)
constant	$O(1)$	une instruction
logarithmique	$O(\log n)$	recherche dichotomique
linéaire	$O(n)$	boucle `for`
linéarithmique	$O(n \log n)$	merge sort, quicksort
quadratique	$O(n^2)$	2 boucles imbriquées, tri par insertion, **à bulles**
cubique	$O(n^3)$	3 boucles imbriquées
exponentiel	$O(b^n)$, $b > 1$	recherche exhaustive
factoriel	$O(n!)$	toutes les permutations



n	$\log_2 n$	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3,3 µs	33 µs	100 µs	1 ms	1,0 ms	3,6 s
20	4,3 µs	86 µs	400 µs	8 ms	1,05 s	771 siècles
30	4,9 µs	147 µs	900 µs	27 ms	17,9 min	$8 \cdot 10^{16}$ siècles
40	5,3 µs	213 µs	1,6 ms	64 ms	12,7 jours	$3 \cdot 10^{32}$ siècles
60	5,9 µs	354 µs	3,6 ms	216 ms	365 siècles	$3 \cdot 10^{66}$ siècles

Un pas = 1 µs (diapo 9, recalculé). Triple boucle `count++` : N^3 pas, soit 1 s pour $N = 100$, 11,6 jours pour 10^4, 317 siècles pour 10^6. DFT en $O(n^2)$, FFT en $O(n \log_2 n)$.

Trois cases fausses dans la diapo 9

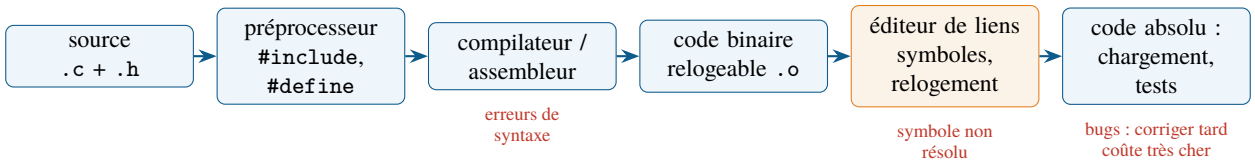
La colonne « $\log_2 n$ » des lignes 10 et 20 est calculée avec ln : 2,3 et 3,0 µs au lieu de 3,3 et 4,3 µs (idem $n \log_2 n$: 23 et 60 au lieu de 33 et 86 µs). Et 2^{60} µs = 365 siècles, pas 266. Les conclusions ne changent pas : au-delà de n^3, la machine ne sauve rien.

Exemple 2 (diapo 13) = le tri à bulles de l'ex. 1.5. La diapo pose la question sans corrigé ; voici une convention *proposée*, au même grain que l'exemple 1 : `s := 0` et le test `until` coûtent t chacun ; chaque tour de boucle coûte $4t$ (incrément, deux accès, comparaison) ; chaque échange coûte $3t$ (hypothèse de la diapo) + $2t$ pour `s := s+1`. Avec P passes et I permutations :

$$c_t = P\,(4N - 2)\,t + 5\,I\,t \qquad \text{trié} : (4N - 2)\,t = O(N) \qquad \text{ordre inverse} : \frac{N(13N-9)}{2}\,t = O(N^2)$$

Pour $N = 10$: $38\,t$ (trié) contre $605\,t$ (inverse). Les constantes dépendent de la convention, pas l'ordre de grandeur. Formules vérifiées par simulation (tous les N de 2 à 8, tirages aléatoires à graine fixe).

3. Langage C : de la source à l'exécutable, types et expressions



Programme : commentaires, directives (`#include <stdio.h>`), variables globales, fonction `main` (nom obligatoire ; `int main(void)` ou `int main(int argc, char *argv[])`), `return 0;`. Fichiers `.c` (code) et `.h` (types et prototypes publics).

Constantes : entières `27`, `0x1A` (hexa), `032` (octal !) ; caractère `'A'`, `'\n'`, `'\0'` ; flottante `45.87e-2` (le point est obligatoire) ; chaîne `"abc"` = 4 octets (le compilateur ajoute `'\0'`). Les constantes binaires `0b1010` de la diapo sont une *extension* de gcc (standard seulement en C23 : `-pedantic` avertit).

Constantes symboliques : `#define MAX (0x7fff)` (substitution de texte : parenthéser !) ou `enum { eBLEU, eJAUNE = 10, eVERT, eROUGE = 30, eBLANC };` → 0, 10, 11, 30, 31 (la numérotation reprend à la dernière valeur imposée). Variable en lecture seule : `uint16_t const cNbr = 10;`.

Conversions : *implicites* dans une expression (`char`, `short` → `int` ; `int` avec `double` → `double`) ; *explicites* par cast `(type)x`. `typedef` crée un nom de type (portabilité : `<stdint.h>`).

Type (PC, diapo 22)	Taille	Plage
`signed char` / `int8_t`	8 bits	−128 … 127
`unsigned char` / `uint8_t`	8 bits	0 … 255
`bool` (C99, `stdbool.h`)	8 bits	0 (false), 1 (true)
`short` / `int16_t`	16 bits	−32 768 … 32 767
`unsigned short`	16 bits	0 … 65 535
`int`, `long` / `int32_t`	32 bits	±2 147 483 647
`unsigned int` / `uint32_t`	32 bits	0 … 4 294 967 295
`long long` / `int64_t`	64 bits	$\approx \pm 9{,}2 \cdot 10^{18}$
`unsigned long long`	64 bits	$0 \ldots 1{,}8 \cdot 10^{19}$
`float` (IEEE 754)	32 bits	$\pm 3{,}4 \cdot 10^{38}$
`double` (IEEE 754)	64 bits	$\pm 1{,}8 \cdot 10^{308}$

La taille de `int` dépend du processeur (note (3) de la diapo) : d'où `<stdint.h>`. Minimum exact de `int32_t` : −2 147 483 648.

Opérateurs, priorité et associativité (diapo « Opérateurs (III) »)

Prio.	Opérateurs	Assoc.
16	`a[k] f(...) . -> x++ x--`	→
15	`++x --x sizeof ~ ! -x &x *p`	←
14	cast `(type)`	←
13	`* / %`	→
12 / 11	`+ - / << >>`	→
10 / 9	`< <= > >= / == !=`	→
8 / 7 / 6	`& / ^ / |` (bit à bit)	→
5 / 4	`&& / ||` (logiques)	→
3	`? :`	←
2	`= += -= *= &= <<=` etc.	←
1	`,`	→

À retenir : les comparaisons passent *avant* `&`, `^`, `|` (parenthéser `(x & masque) == 0`) ; `&&` et `||` **court-circuitent** (le 2e opérande n'est pas évalué si le 1er suffit) ; le C ne teste **aucun dépassement** ; `%` est interdit sur les flottants ; `>>` sur un signé dépend du compilateur.

Incrément (diapo « Variables (VIII) ») : `x = n++` rend l'*ancienne* valeur, `x = ++n` la nouvelle. `n = 17; x = n++;` ⇒ $x = 17$, $n = 18$; `n = 8; s[n++] = 8;` ⇒ `s[8]` = 8, $n = 9$; puis `n++; s[--n] = 6;` ⇒ $n = 9$, `s[9]` = 6.

Affectation composée : `v += 10 * (5 + w);` équivaut à `v = v + (10 * (5 + w));`.

Opérateur ternaire : `return (a > b) ? a : b;` remplace un `if-else` (attention aux points d'arrêt du débogueur).

Ex. 1.1 d) et e) — types, valeurs, ordre d'évaluation

N°	Expression	Type	Valeur
1	`d/2`	`double`	2.6
2	`s*4`	`int`	32
3	`(b+s) > (5*f)`	`int`	1
4	`i/4 + d`	`double`	7.2
5	`1/3`	`int`	0
6	`b&6`	`int`	4

`unsigned char b = 5; short s = 8; int i = 9; float f = 2.5; double d = 5.2;` — types vérifiés par `_Generic`. Pour 3 : 13 > 12,5 ; pour 4 : 9/4 = 2 (entier) puis 2 + 5,2 ; `1.0/3` = 0,333 333.

N°	Évalué comme	a b j k
1	`int j = 7, k = 4; bool a = true, b = true;`	1 1 7 4
2	`b = (a != true);`	1 0 7 4
3	`a = ((a && b) && true);`	0 0 7 4
4	`a = ((7 >= 3) | (7 == 3));`	1 0 7 4
5	`b = ((k++ >= 5) || (j-- == 7));`	1 1 6 5
6	`k = k++ + ++j;`	indéfini

Ligne 5 : `4 >= 5` est faux, donc `j-- == 7` est évalué (7 == 7 vrai, puis $j = 6$) ; `k++` a déjà porté k à 5.

Expression 6 : comportement indéfini

`k = k++ + ++j;` modifie `k` deux fois sans point de séquence : la norme C ne fixe pas le résultat (`gcc -Wall` avertit « operation on 'k' may be undefined »). La sortie de l'énoncé ($k = 12, j = 7$) n'est qu'*un* résultat possible : ne jamais écrire une telle ligne.

```c
#include <stdbool.h>

/* Ex. 1.3 / 1.4 b) : 'a' = 0x61, 'A' = 0x41 (bit 5 = 0x20) */
char toUpperAscii(char c)
{
    if ((c >= 'a') && (c <= 'z')) {
        c &= ~0x20;       /* ET avec ~masque : efface le bit 5 */
    }
    return c;
}

/* Ex. 1.4 d) : ternaire  condition ? siVrai : siFaux */
char const *boolText(bool value)
{
    return value ? "true" : "false";
}
```

Ex. 1.4 avec `cVarA = 12` (1100) et `cVarB = 10` (1010) : `12|10` = 14 (1110), `12&10` = 8 (1000), `12^10` = 6 (0110), `~12` = −13 (complément à 1 d'un `int` signé : $-x-1$).

Logique : toute valeur non nulle est vraie : `12 || 10` et `12 && 10` valent 1, `!(12 == 10)` aussi.

Majuscule (Ex. 1.3 et 1.4 b) : en ASCII, `'a'` = 0x61 et `'A'` = 0x41 ne diffèrent que par le bit 5 (0x20) ; `c &= ~0x20` l'efface. On teste d'abord `'a' <= c <= 'z'` (en C : `(c >= 'a') && (c <= 'z')`), sinon on abîmerait les chiffres et la ponctuation.

4. Structures de contrôle et fonctions

Boucles : `do {...} while (cond);` s'exécute *au moins une fois* (copie de chaîne jusqu'au `'\0'`) ; `while (cond) {...}` teste d'abord ; `for (init; cond; pas) {...}` regroupe les trois (variable déclarée dans `init` depuis C99). Toujours modifier dans le corps ce que teste la condition. Boucle sans fin : `while (1)` ou `for (;;)` (programme principal d'un système embarqué, reset par watchdog).

Conditions : `if-else` ; `switch (expr) { case CONST: ... break; default: ... }` compare à des **constantes entières** ; sans `break`, l'exécution « tombe » dans le `case` suivant (à commenter : `// Fall through`). Sauts : `break`, `continue` (`goto` à éviter).

Fonctions : la *déclaration* (prototype, signature) `int add(int, int);` suffit au compilateur ; la *définition* contient le corps. Trois façons : prototype avant `main` et corps après ; corps avant `main` ; prototype dans un `.h` protégé par `#ifndef X_H / #define X_H / #endif` et corps dans un `.c`. `void f(void)` = procédure ; `static` devant une fonction la rend **privée** au fichier.

Passer l'information : variables globales (facile, mais effets de bord) ; **arguments par valeur** (copie) + `return` ; résultat **par référence** via un pointeur (`void addAndModulo2(uint32_t a, uint32_t b, uint32_t *pResult)`, appel avec `&resultOp4`).

Variables : `[classe] TYPE [qualificatif] nom [= init];`. Classes : `auto` (défaut), `static` (globale : visible dans le seul fichier ; locale : **garde sa valeur** entre les appels), `extern` (définie dans un autre fichier, résolue à l'édition de liens), `register`. Qualificatifs : `const` (lecture seule), `volatile` (peut changer hors du programme : interruption, registre).

Déclaration (type et nom) ≠ **définition** (réserve la mémoire, peut initialiser) ≠ **affectation** (dans le code). Une variable locale non initialisée contient n'importe quoi.

Portée, du plus large au plus étroit : programme, fichier (`static`), fonction, bloc (C99). Une variable d'un bloc intérieur **masque** celle du bloc extérieur de même nom.

```c
/* Ex. 1.6 : passage par valeur, résultat par return */
double celsiusToFahrenheit(double celsius)
{
    return celsius * 1.8 + 32.0;
}

/* Diapo « Fonctions (V) » : x et y sont des COPIES */
void swapByValue(int x, int y)
{
    int const tmp = x;
    x = y;
    y = tmp;                /* l'appelant ne voit rien */
}

/* Passage « by reference » : on reçoit les ADRESSES */
void swapByReference(int *pX, int *pY)
{
    int const tmp = *pX;
    *pX = *pY;
    *pY = tmp;              /* l'appelant voit l'échange */
}

/* static local : garde sa valeur entre les appels */
int getCount(void)
{
    static int count = 0;  /* initialisée une seule fois */
    count++;
    return count;
}
```

Ex. 1.6 — convertisseur : une fonction par tâche, des **constantes** pour les bornes.

$$T_F = 1{,}8\,T_C + 32 \qquad (T_F \text{ en °F},\ T_C \text{ en °C})$$

De −20 à 30 °C par pas de 1 : **51 lignes** ; −20 → −4,0, −19 → −2,2, 10 → 50,0, 29 → 84,2, 30 → 86,0 °F (et −40 °C = −40 °F). La sortie de l'énoncé (`+29.0`) correspond au format `%+.1f`. Sur la console Windows, le symbole ° s'obtient avec le code 0xF8 (248).

Échange : `swapByValue(x, y)` ne change rien chez l'appelant (diapo « Fonctionne ? » : non) ; `swapByReference(&x, &y)` échange bien 13 et 15.

5. Tableaux, structures, unions (Ex. 1.7, 1.8)

Tableau : éléments d'un même type, contigus. `int32_t t[] = {1, 3, 5};` ; indices de **0 à** $n-1$; le C ne contrôle pas les bornes. Nombre d'éléments sans le noter en dur : `sizeof t / sizeof t[0]`. 2D : `int32_t m[4][2]` ; tableau de chaînes : `char const cFruits[][NameLen] = {"orange", ...}`. Le **nom du tableau** est l'adresse de son 1er élément.

Structure : regroupe des données liées, de *types différents* ; `typedef struct {...} Student_t;` (recommandé) ; accès `s.membre`, imbriqué `s.homeAddress.city` ; initialisation `{"Linus", "Torvalds", eClass_E1a, {...}}` ; l'affectation `a = b;` copie *tous* les champs (tableaux internes compris). **Champ de bits** : `uint8_t P1OUT0 : 1;` donne accès à un bit d'un registre (ordre des bits dépendant du compilateur).

Union : tous les membres partagent la *même* mémoire ; taille = celle du plus grand. Usages : une donnée de type variable dans le temps, conversion de type, registre vu en 16 bits ou en deux octets (`PAOUT` du MSP430).

```c
#include <limits.h>
#include <stdio.h>
#include <time.h>

#define DATE_FORMAT "%02d.%02d.%04d %02d:%02d:%02d"

/* Ex. 1.7 c) : texte + struct tm passée par valeur (copie) */
void showTime(char const *text, struct tm t)
{
    printf("%s" DATE_FORMAT "\n", text, t.tm_mday, t.tm_mon + 1,
           t.tm_year + 1900, t.tm_hour, t.tm_min, t.tm_sec);
}

/* d) tm_wday (0 = dimanche) -> nom du jour ;
      static : le tableau survit au return */
char const *weekDayName(int weekDay)
{
    static char const *const cNames[] = {"Sunday",
        "Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday"};
    if (weekDay < 0 || weekDay > 6) {
        return "?";                          /* hors table */
    }
    return cNames[weekDay];
}

/* b) dernière seconde d'un time_t 32 bits signé :
      INT_MAX = 2^31 - 1 */
struct tm lastSecondOfTime32(void)
{
    time_t const cMax = (time_t)INT_MAX;
    return *localtime(&cMax);   /* localtime rend un POINTEUR */
}
```

Ex. 1.7 — `struct tm` (`<time.h>`) : `tm_mon` de 0 à 11 (+1 à l'affichage), `tm_year` compté depuis 1900 (+1900), `tm_wday` de 0 = dimanche à 6, `tm_yday` de 0 à 365. `time(NULL)` rend le temps courant (secondes depuis 1970) ; `localtime(&t)` rend un **pointeur** vers une `struct tm` interne, d'où `*localtime(&t)`.

Ex. 1.7 b) — dépassement d'un `time_t` 32 bits signé : $2^{31}-1$ = 2 147 483 647 s (`INT_MAX` de `<limits.h>`) après le 1.1.1970 :

> 19.01.2038 03:14:07 UTC = 04:14:07 à Fribourg (un mardi)

Sur ce PC (gcc MinGW, UCRT), `time_t` fait 64 bits : le problème « Y2038 » touche les `time_t` 32 bits (embarqué, anciens systèmes).

Piège de la note 1.7 : `DATE_FORMAT` commence par `%02d` ; pour afficher le texte devant, écrire `"%s" DATE_FORMAT` (concaténation de littéraux), jamais passer un `char *` à un `%d`.

Ex. 1.8 — gestion d'adresses : structure imbriquée, `enum` pour la classe, `typedef` de chaînes, tableau de 30 étudiants mis à zéro par `memset(studentsGE, 0, sizeof studentsGE)`, ajout, tri à bulles sur le nom de famille (§ 8). Résultat trié (testé) : Bykov, Lüthi, Sauber, Stallman, Torvalds, Wawrinka.

```c
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>

/* Ex. 1.8 : constantes et types (diapo « Structures (I) ») */
enum { eNameLen = 20, eAddressLen = 40, eMaxStudents = 30 };
typedef char Name_t[eNameLen];
typedef char AddressName_t[eAddressLen];
typedef enum { eClass_E1a, eClass_E1bd, eClass_E2a,
    eClass_E2bd, eClass_E3_EL, eClass_E3_EN } Class_t;

typedef struct {
    AddressName_t street;
    int houseNbr;
    int zipCode;
    AddressName_t city;
} Address_t;

typedef struct {
    Name_t firstName;
    Name_t lastName;
    Class_t currentClass;
    Address_t homeAddress;          /* structure imbriquée */
} Student_t;
```

```c
/* c) affichage de tout le tableau, message si vide */
void printStudents(Student_t const table[], size_t count)
{
    static char const *const cClass[] = {"E-1a", "E-1bd",
        "E-2a", "E-2bd", "E-3EL", "E-3EN"};
    if (count == 0u) {
        printf("No student in table available yet!\n");
        return;
    }
    for (size_t i = 0u; i < count; i++) {
        Student_t const *const pS = &table[i];
        Address_t const *const pA = &pS->homeAddress;
        printf("%u : %s %s, %s, %s %d, %d-%s\n", (unsigned)i,
               pS->firstName, pS->lastName,
               cClass[pS->currentClass], pA->street,
               pA->houseNbr, pA->zipCode, pA->city);
    }
}
```

```c
/* d) ajout par copie de la structure ; *pCount mis à jour */
bool addStudent(Student_t table[], size_t *pCount,
                Student_t const *pNew)
{
    if (*pCount >= eMaxStudents) {
        return false;               /* tableau plein */
    }
    table[*pCount] = *pNew;     /* copie de tous les champs */
    (*pCount)++;
    return true;
}

/* g) tri à bulles sur le nom (strcmp < 0 : avant) */
void sortByLastName(Student_t table[], size_t count)
{
    bool swapped;
    do {
        swapped = false;
        for (size_t i = 1u; i < count; i++) {
            if (strcmp(table[i].lastName,
                       table[i - 1u].lastName) < 0) {
                Student_t const tmp = table[i];
                table[i] = table[i - 1u];
                table[i - 1u] = tmp;
                swapped = true;
            }
        }
    } while (swapped);
}
```

`pS->homeAddress` est une structure : on prend son adresse (`pA`) pour alléger les accès. `strcmp(a, b)` rend un nombre < 0, 0 ou > 0 selon l'ordre des *octets* : il compare des codes, pas l'ordre alphabétique français (« Émile » passerait après « Zoé »).

6. Pointeurs (Ex. 1.11)

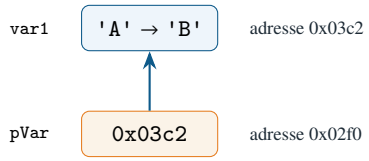
Définition : un pointeur est une variable qui contient l'**adresse** d'une donnée (variable, structure, fonction…). Usages : rendre des résultats depuis une fonction, créer des objets dynamiques, accéder aux **registres** matériels. Déclaration : `TYPE *pVariable;`.

Adresse : `pVar = &var1;`. **Déréférenciation** : `*pVar = 'B';` écrit dans `var1`. Il faut lui donner une adresse valide *avant* : déréférencer `NULL` ou un pointeur non initialisé est un **comportement indéfini** (plantage).

Structure : `pPoint->x` ≡ `(*pPoint).x` (les parenthèses sont nécessaires : `.` passe avant `*`).

Tableaux : `ptbl = tbl;` ≡ `ptbl = &tbl[0];`; `p + 1` avance de `sizeof(*p)` octets : avec `uint16_t a[4]` en 0x3C6, `a + 1` vaut 0x3C8; `a[i]` ≡ `*(a + i)`.

Pointeur `void *` : générique; le convertir avant de le déréférencer : `*((char *)pVar)`.




```c
char *pVar = &var1;
*pVar = 'B';
```

Appel (diapo 11)	Effet chez l'appelant
`foo1(j)`, `i = 2;`	`j` reste 1 (copie)
`foo2(&j)`, `*pi = 2;`	`j` vaut 2
`foo3(ptr)`, `pi = malloc(...)`	`ptr` reste `NULL`
`foo4(&ptr)`, `*ppi = malloc(...)`	`ptr` reçoit l'adresse

Pour modifier un pointeur de l'appelant : `TYPE **pp`.

Pointeurs de fonction (diapos 12–14)

```c
#include <stddef.h>
#include <stdint.h>

/* Diapos 12-14 : pointeur vers int16_t f(int16_t, int16_t) */
typedef int16_t (*Function_t)(int16_t, int16_t);

int16_t add(int16_t a, int16_t b) { return (int16_t)(a + b); }
int16_t sub(int16_t a, int16_t b) { return (int16_t)(a - b); }

/* tableau de pointeurs de fonction */
static Function_t const cOperations[] = {add, sub};

/* fonction qui RENVOIE un pointeur de fonction */
Function_t getFunction(size_t index)
{
    return (index < sizeof cOperations / sizeof cOperations[0])
           ? cOperations[index] : NULL;     /* hors table */
}
```

Syntaxe : `TypeRetour (*nom)(arguments);` — les parenthèses autour de `*nom` sont indispensables (sinon c'est une fonction qui renvoie un pointeur). Le `typedef` rend tout lisible.

Affectation : `f = add;` ou `f = &add;` (même adresse, testé). **Appel** : `f(10, 20)` ou `(*f)(10, 20)` → 30. Tableau : `cOperations[1](10, 20);` fonction qui rend un pointeur de fonction : `getFunction(1)(10, 20)` → −10.

Usage : appel *dynamique* (choisir l'opération à l'exécution), callbacks, critère de tri passé à une fonction de tri — c'est la partie e) de l'ex. 1.11.

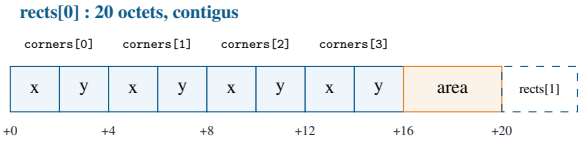
Ex. 1.11 — rectangles, adresses et tri via un pointeur de fonction

```c
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

/* Ex. 1.11 a) : point 16 bits ; rectangle = 4 points + aire */
enum { eNbrOfCorners = 4 };

typedef struct {
    int16_t x;
    int16_t y;
} Point_t;                              /* 2 x 2 octets */

typedef struct {
    Point_t corners[eNbrOfCorners];     /* dans l'ordre du tour */
    int32_t area;                       /* calculée en d) */
} Rectangle_t;
```



c) Disposition mémoire (testée par `offsetof`) : un `Point_t` fait 4 octets (y à +2), les 4 coins occupent 16 octets, `area` est à l'offset 16 et un `Rectangle_t` fait **20 octets** (0x14). Le tableau est **contigu** (points → rectangles → tableau) : coin k du rectangle r à l'adresse base + $20\,r + 4\,k$.

```c
/* b) affichage : pointeur vers const (lecture seule) */
void printRectangle(Rectangle_t const *pRect)
{
    for (size_t k = 0u; k < eNbrOfCorners; k++) {
        printf("[%2d,%2d]", pRect->corners[k].x,
               pRect->corners[k].y);
    }
    printf("\n");
}

/* d) côtés parallèles aux axes : coins 0 et 2 opposés */
void computeArea(Rectangle_t *pRect)
{
    int32_t const w = pRect->corners[2].x - pRect->corners[0].x;
    int32_t const h = pRect->corners[2].y - pRect->corners[0].y;
    pRect->area = (w < 0 ? -w : w) * (h < 0 ? -h : h);
}   /* pRect->area  équivaut à  (*pRect).area */
```

d) et e) Rectangles d'exemple (seul le 1er vient de l'énoncé) : surfaces 100, 400 et 180 ; après le tri appelé par la variable `sortFct`, l'ordre est [0,0], [0,5], [10,10], celui de la sortie e) de l'énoncé.

```c
/* e) tri à bulles par aire (structures copiées entières) */
void sortByArea(Rectangle_t table[], size_t n)
{
    size_t swaps;
    do {
        swaps = 0u;
        for (size_t i = 1u; i < n; i++) {
            if (table[i].area < table[i - 1u].area) {
                Rectangle_t const tmp = table[i];
                table[i] = table[i - 1u];
                table[i - 1u] = tmp;
                swaps++;
            }
        }
    } while (swaps != 0u);
}

typedef void (*SortFunction_t)(Rectangle_t table[], size_t n);

/* d) + e) : aires, puis tri via un pointeur de fonction */
void processRectangles(Rectangle_t table[], size_t n)
{
    /* = sortByArea ou &sortByArea : même adresse */
    SortFunction_t const sortFct = sortByArea;
    for (size_t i = 0u; i < n; i++) {
        computeArea(&table[i]);   /* adresse de l'élément i */
    }
    sortFct(table, n);        /* ou (*sortFct)(table, n) */
}
```

7. Récursion (Ex. 1.2, 1.10, 1.14)

Définition : une fonction est récursive si elle s'appelle elle-même, directement ou par une suite d'appels (`func1` → `func2` → … → `func1`). Une **condition d'arrêt** est indispensable : sans elle, la pile déborde (« stack overflow »).

Quand l'utiliser : un ou plusieurs **cas simples** ont une solution triviale, et les autres se **reformulent** en problèmes plus proches de ces cas. Schéma : `if simpleCase solve else reformulate using recursion`.

Diviser pour régner : un problème de taille n se découpe en un problème de taille 1 et un de taille $n-1$ (ou en deux moitiés), jusqu'au cas simple.

Exemples de la diapo : $n! = n \cdot (n-1)!$ avec $0! = 1$; Fibonacci $f(n) = f(n-1) + f(n-2)$, $f(1) = f(2) = 1$; algorithmes de tri ; multiplication par additions : `mult(n, m)` rend `m` si $n = 1$, sinon `m + mult(n-1, m)`. Pile : `mult(3,5)` → `mult(2,5)` → `mult(1,5)` rend 5, puis 5 + 5, puis 5 + 10 = 15.

```c
#include <stdint.h>

/* Ex. 1.2 a) itération */
uint64_t factorialIteration(uint32_t n)
{
    uint64_t result = 1u;
    for (uint32_t i = 2u; i <= n; i++) {
        result *= i;
    }
    return result;
}

/* b) récursion : n! = n * (n-1)! */
uint64_t factorialRecursion(uint32_t n)
{
    if (n <= 1u) {
        return 1u;          /* cas de base : 0! = 1! = 1 */
    }
    return n * factorialRecursion(n - 1u);
}
```

Ex. 1.2 : trois algorithmes pour le même résultat, de 0! à 20!, affichés par la même boucle de `main` (comparaison à conditions égales). Déroulé récursif : `factorialRecursion(3)` = 3·2·1 = 6, les appels s'empilent jusqu'au cas de base puis se dépilent.

```c
/* c) LUT : le préprocesseur substitue,
      le compilateur évalue les produits */
#define F1  1ull
#define F2  (F1 * 2u)
#define F3  (F2 * 3u)
#define F4  (F3 * 4u)
#define F5  (F4 * 5u)
#define F6  (F5 * 6u)
#define F7  (F6 * 7u)
#define F8  (F7 * 8u)
#define F9  (F8 * 9u)
#define F10 (F9 * 10u)
#define F11 (F10 * 11u)
#define F12 (F11 * 12u)
#define F13 (F12 * 13u)
#define F14 (F13 * 14u)
#define F15 (F14 * 15u)
#define F16 (F15 * 16u)
#define F17 (F16 * 17u)
#define F18 (F17 * 18u)
#define F19 (F18 * 19u)
#define F20 (F19 * 20u)

static uint64_t const cFactLut[] = {
    1ull, F1, F2, F3, F4, F5, F6, F7, F8, F9, F10,
    F11, F12, F13, F14, F15, F16, F17, F18, F19, F20};
/* nombre d'éléments : 21 (de 0! à 20!) */
enum { eFactLutLen = sizeof cFactLut / sizeof cFactLut[0] };

uint64_t factorialLut(uint32_t n)
{
    /* 0 : hors table (21! déborde de 64 bits) */
    return (n < eFactLutLen) ? cFactLut[n] : 0u;
}
```

LUT (*look-up table*) : au lieu de calculer, on lit le résultat rangé d'avance, par son index. Le préprocesseur ne fait que substituer `F20` par `(F19 * 20u)` ; c'est le **compilateur** qui calcule les produits constants, une fois pour toutes. `static const` : un seul exemplaire, non modifiable.

Nombre d'éléments : `sizeof cFactLut / sizeof cFactLut[0]` = 168/8 = 21, sans l'écrire en dur ; on refuse tout index ≥ 21 (lecture hors tableau sinon).

Tailles : 12! = 479 001 600 tient sur 32 bits, 13! = 6 227 020 800 non ; 20! = 2 432 902 008 176 640 000 est la dernière sur 64 bits. 21! calculé en `uint64_t` rend 14 197 454 024 290 336 768, faux (modulo 2^{64}). Affichage : `%llu`.

En embarqué, la LUT convient quand l'ensemble des résultats est fini et connu d'avance : lecture rapide et *prévisible*, payée en mémoire.

Ex. 1.2	a) itération	b) récursion	c) table (LUT)
Principe	boucle `for`, produit accumulé	cas de base + $n \cdot (n-1)!$	résultats rangés d'avance
Mémoire	une variable	une trame de pile par appel	21 valeurs `uint64_t` (168 octets)
Temps	$O(n)$	$O(n)$, plus lent (appels)	$O(1)$: une lecture
Limite	dépassement au-delà de 20!	idem	bornée par la table : $n \leq 20$

Tours de Hanoï (Ex. 1.10)

```c
#include <stdint.h>
#include <stdio.h>

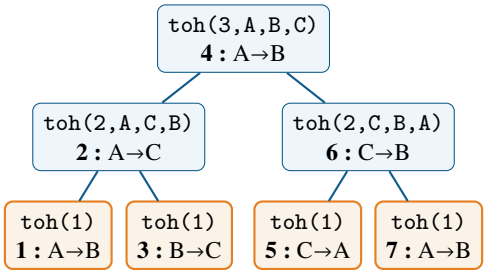
static void moveDisk(uint32_t disk, char from, char to,
                     uint32_t *pMoves)
{
    printf("Move disk %u from %c to %c\n",
           (unsigned)disk, from, to);
    (*pMoves)++;        /* b) compteur sans variable globale */
}

/* Ex. 1.10 : pseudo-code de l'énoncé ; *pMoves compte */
void toh(uint32_t disk, char source, char dest, char aux,
         uint32_t *pMoves)
{
    if (disk == 1u) {                          /* cas simple */
        moveDisk(1u, source, dest, pMoves);
    } else {
        toh(disk - 1u, source, aux, dest, pMoves); /* -> aux */
        moveDisk(disk, source, dest, pMoves);    /* le grand */
        toh(disk - 1u, aux, dest, source, pMoves); /* -> dest */
    }
}
```

Règles : un seul disque à la fois, jamais un grand sur un petit. Déplacer n disques = déplacer $n-1$ disques sur l'aiguille auxiliaire, le plus grand sur la destination, puis les $n-1$ par-dessus.

$$M(n) = 2\,M(n-1) + 1, \quad M(1) = 1 \quad \Rightarrow \quad M(n) = 2^n - 1$$

b) Le compteur passe par **pointeur** (`uint32_t *pMoves`) : pas de variable globale.



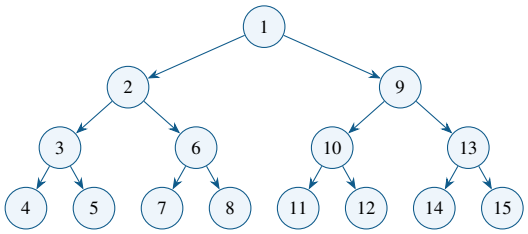
Arbre d'appels de `toh(3, 'A', 'B', 'C')` : les numéros donnent l'ordre des 7 déplacements, identique à la sortie de l'énoncé (« Move disk 1 from A to B », …). **64 disques** : $2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$ coups, soit $3{,}5 \cdot 10^{13}$ ans à 1 coup par minute, $5{,}8 \cdot 10^{8}$ ans à 1000 coups/s et encore **585 ans** à 10^9 coups/s.

Alerte téléphonique (Ex. 1.14)

```c
#include "liste-telephonique.h"

/* Ex. 1.14 : list[0] = moi (déjà informé) ;
   list[1..size-1] = les personnes que je dois prévenir */
void phoneAlarm(ListRecord_t const list[], size_t size)
{
    if (size <= 1u) {          /* personne à prévenir : rien */
        return;
    }
    size_t const rest = size - 1u;      /* ma liste, sans moi */
    size_t const half1 = rest / 2u;     /* coupée en deux */
    size_t const half2 = rest - half1;
    if (half1 > 0u) {
        showCalledPerson(&list[1]);  /* 1re, moitié 1 ... */
        phoneAlarm(&list[1], half1); /* ... prévient sa moitié */
    }
    showCalledPerson(&list[1u + half1]); /* 1re, moitié 2 */
    phoneAlarm(&list[1u + half1], half2);
}
```

Lecture de la consigne : `list[0]` est la personne qui appelle (Donald au départ, déjà informé) ; elle coupe *le reste* de sa liste en deux, appelle la 1re personne de chaque moitié et lui confie cette moitié. Le type `ListRecord_t` et `showCalledPerson` sont donnés par l'énoncé (`extraits/liste-telephonique.h`).



15 entrées = $1+2+4+8 = 2^4 - 1$: arbre complet, **14 appels**, 3 niveaux ; ordre d'affichage 2, 3, …, 15 (Donald appelle Huey puis Scrooge).

14 entrées : les moitiés deviennent inégales (6 et 7) ; **13 appels**, toujours 3 niveaux, personne n'est oublié, mais Dewey (3) n'appelle plus que Louie (4).

Deux pièges de l'énoncé 1.14

(1) Le numéro de Gyro, `"877-764-2532-999"`, a **16 caractères** dans un `char[16]` : le C l'accepte mais n'y met **pas** de `'\0'`, et `printf("%s")` lit au-delà du tableau. Remède : `%.16s` (fait dans le banc de test) ou un tableau de 17. (2) Le brouillon de `Code/` calcule `halfright = (size - halfleft - 1)/2` et n'appelle que 2, 3, 4, 5, 7, 10 et 11 : la 2e moitié doit être *le reste* (reste – moitié$_1$), pas une nouvelle moitié.

8. Tri à bulles (Ex. 1.5, réutilisé en 1.8 et 1.11)

```c
#include <stddef.h>
#include <stdint.h>

/* Ex. 1.5 : pseudo-code de l'énoncé (repeat ... until s = 0).
   Renvoie le nombre total de permutations. */
uint32_t bubbleSort(int16_t table[], size_t n)
{
    uint32_t permutations = 0u;
    uint32_t s;
    do {
        s = 0u;                                 /* s := 0 */
        /* for i = 2 to N  (indices C : 1 à n-1) */
        for (size_t i = 1u; i < n; i++) {
            if (table[i] < table[i - 1u]) {
                int16_t const tmp = table[i];  /* swap : 3 t */
                table[i] = table[i - 1u];
                table[i - 1u] = tmp;
                s++;                            /* s := s + 1 */
            }
        }
        permutations += s;
    } while (s != 0u);                          /* until s = 0 */
    return permutations;
}
```

Principe : on compare chaque paire voisine et on l'échange si elle est dans le mauvais ordre ; chaque passe pousse le plus grand restant en fin de tableau ; on s'arrête après une passe **sans échange** (`s = 0`).

Nombre de permutations = nombre d'**inversions** (paires $i < j$ avec $t_i > t_j$) : chaque échange de voisins en corrige exactement une.

Séquence (10 nombres)	Permut.	Passes
énoncé : 28 3 45 1 5 17 33 32 42 2	20	9
déjà trié	0	1
ordre inverse	$45 = \frac{10\cdot 9}{2}$	10
plus petit à la fin (2 … 10, 1)	9	10
plus grand au début (10, 1 … 9)	9	2

Un petit élément en fin de tableau ne recule que d'une case par passe (« tortue ») ; un grand au début traverse tout en une passe (« lièvre »). **Complexité** : $O(n^2)$ dans le pire cas, $O(n)$ si déjà trié (grâce au test `s = 0`).

Généraliser (Ex. 1.8 et 1.11) : on garde la boucle, on change le **critère** (`strcmp(...) < 0` pour un nom, `.area` pour une surface) et l'échange copie la structure entière ; le critère peut même être passé en **pointeur de fonction**.

9. Méthode au TE — quelle question, quel réflexe

On te demande…	Réflexe
le type et la valeur d'une expression	promotions : `char`, `short` → `int` ; avec un `double` → `double` ; deux entiers ⇒ division entière ; une comparaison vaut l'`int` 0 ou 1
l'ordre d'évaluation	parenthéser selon la table des priorités ; `&&`, `||` court-circuitent ; `x++` rend l'ancienne valeur ; deux modifications d'une même variable ⇒ indéfini
le coût d'un algorithme	compter les instructions élémentaires (en t) en fonction de N, garder le terme dominant ⇒ $O(\cdot)$; distinguer meilleur et pire cas
un organigramme ou un Nassi-Shneiderman	séquence, décision (losange / triangle oui-non), répétition (test avant ou après) ; la boucle englobe son corps
une fonction récursive	écrire le **cas de base** d'abord, puis la réduction ; dessiner l'arbre (ou la pile) d'appels ; compter les appels
de modifier une variable de l'appelant	passer son adresse (`TYPE *p`, appel avec `&x`), écrire `*p = ...` ; pour un pointeur : `TYPE **pp`
de traiter un tableau de structures	fonction `(TYPE const *p, size_t n)`, `n = sizeof t / sizeof t[0]`, accès `p->champ` ou `t[i].champ`
la disposition mémoire	`sizeof` de chaque membre, offsets, éléments contigus ; `p + 1` avance de `sizeof(*p)` octets
de trier	pseudo-code `repeat ... until s = 0` ; permutations = inversions ; changer seulement le critère

10. Mémo des pièges

1. Indices d'un tableau : de 0 à $n-1$; le C ne vérifie pas les bornes.
2. `1/3` vaut 0 (division entière) : écrire `1.0/3`.
3. `=` affecte, `==` compare.
4. `&` et `|` (bits) ≠ `&&` et `||` (logiques, court-circuit).
5. `(x & m) == 0` : parenthéser, `==` passe avant `&`.
6. Jamais deux modifications de la même variable dans une expression (`k = k++ + ++j`).
7. Échanger par valeur ne fait rien : passer les adresses.
8. Déréférencer `NULL` ou un pointeur non initialisé : comportement indéfini.
9. Une chaîne de n caractères occupe $n+1$ octets (`'\0'`).
10. `tm_mon` 0–11, `tm_year` depuis 1900, `tm_wday` 0 = dimanche.
11. `localtime` rend un pointeur : `*localtime(&t)`.
12. Récursion sans cas de base : débordement de pile.
13. 13! dépasse 32 bits et 21! 64 bits : `uint64_t`, `%llu`.
14. Ne jamais rendre l'adresse d'une variable locale non `static`.
15. `#define` : parenthéser la valeur.
16. Un tableau passé en paramètre devient un pointeur : `sizeof` ne donne plus sa taille, passer n.
17. Tri à bulles : remettre `s` à 0 à chaque passe.
18. `032` est octal (26), pas décimal.

11. Auto-test (réponses en bas de page)

1. Avec les déclarations de l'ex. 1.1 d), type et valeur de `s*4` et de `i/4 + d` ?
2. Après `int j = 7, k = 4; bool b = k++ >= 5 || j-- == 7;`, que valent `b`, `j`, `k` ?
3. Que valent `~12` et `12 ^ 10` en `int` ?
4. Coût en temps de l'exemple 1 pour $N = 10$? Classe de complexité ?
5. Combien de permutations et de passes le tri à bulles fait-il sur 5, 4, 3, 2, 1 ?
6. Diapo « Variables (VI) » : `getVal` contient `static int i = 2; i++;` puis une boucle `for (int i = 0; i < 10;) { i++; }` et `return i;`. Qu'affiche `main` ?
7. Combien de déplacements pour 10 disques de Hanoï ? Combien de temps pour 64 disques à 10^9 coups/s ?
8. Pourquoi `swapByValue(x, y)` échoue-t-il ? Écrire l'appel correct.
9. `sizeof(Rectangle_t)` ? Adresse du coin 2 du rectangle 1 si le tableau commence en 0x61FECC ?
10. Avec 14 entrées, combien d'appels `phoneAlarm` déclenche-t-il ? Quelqu'un est-il oublié ?

Réponses. 1) `int`, 32 ; `double`, 7.2 (9/4 = 2 en entier). 2) `b` = 1, `j` = 6, `k` = 5 (`k++ >= 5` est faux, donc `j-- == 7` est évalué). 3) −13 ; 6. 4) $41\,t$; $O(N)$. 5) 10 permutations ($5 \cdot 4/2$) en 5 passes. 6) `i=3!` : le `static` local passe de 2 à 3, la boucle utilise sa propre variable `i`. 7) $2^{10} - 1 = 1023$; environ 585 ans. 8) La fonction reçoit des copies ; `swapByReference(&x, &y);`. 9) 20 octets ; 0x61FECC + 20 · 1 + 4 · 2 = 0x61FEE8. 10) 13 appels, personne n'est oublié ; les moitiés sont inégales (6 et 7).